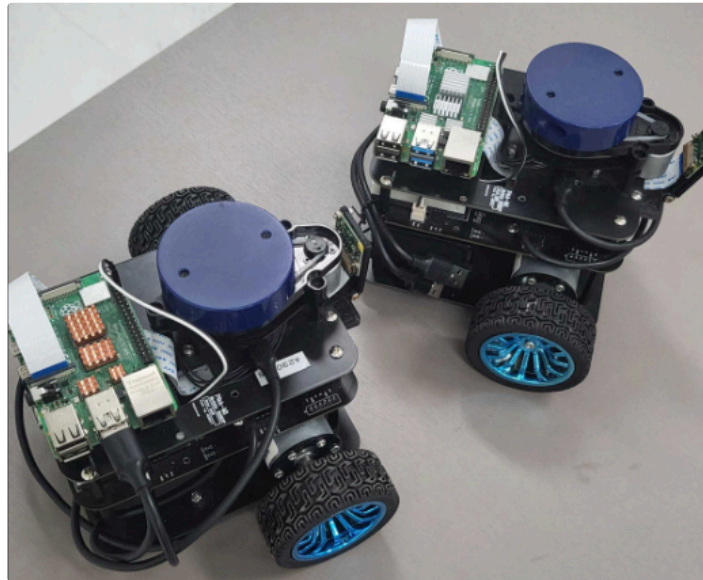


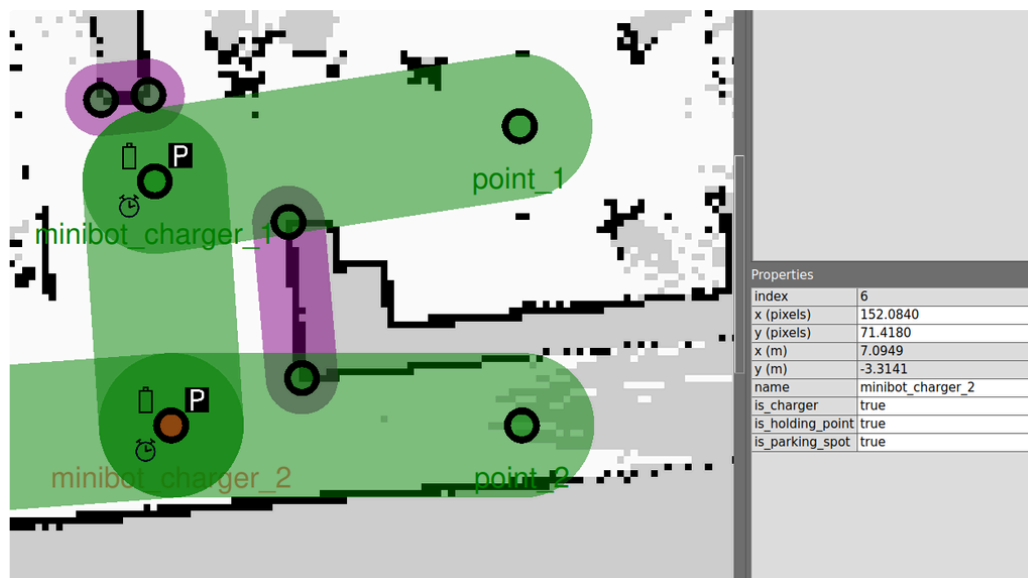
3. 2대 이상의 RMF 시스템 구축

RMF시스템에 같은 Fleet인 minibot을 한개 더 추가하는 방법 [↗](#)

Minibot을 하나 더 다뤄보자. [↗](#)



Traffic editor를 열어서, 다른 minibot이 시작할 지점을 만들자. [링크1](#) [↗](#)



fleet_config에서 방금 만든 vertex로 minibot_2의 시작점을 만들어준다. [링크2](#) [↗](#)

- rmf_demos/config/pinklab_real 에 minibot fleet의 config파일이 있다.

```
# minibot CONFIG =====
robots:
  minibot_1:
    robot_config:
      max_delay: 10.0 # allowed seconds of delay of the current itinerary be
    rmf_config:
      robot_state_update_frequency: 1.0
      charger:
        waypoint: "minibot_charger_1"
      start:
        map_name: "L1"
        waypoint: "minibot_charger_1"
        orientation: 0.0 # radians
  minibot_2:
    robot_config:
      max_delay: 10.0 # allowed seconds of delay of the current itinerary be
    rmf_config:
      robot_state_update_frequency: 1.0
      charger:
        waypoint: "minibot_charger_2"
      start:
        map_name: "L1"
        waypoint: "minibot_charger_2"
        orientation: 0.0 # radians
```

Navigation2시작시 Free Fleet client설정을 바꿔서 이름을 입력인자로 받을 수 있게 변경하자. [링크3](#)

- Navigation2 시작파일은 minibot_navigation2/launch/bringup_launch.xml 에 있다.

```
<launch>
  <set_env name="RCUTILS_LOGGING_BUFFERED_STREAM"
  <arg name="namespace" default="" />
  <arg name="map" default="${find-pkg-share minibot_navigation2}/maps/pinklab_real/pinklab_L1.yaml" />
  <arg name="use_sim_time" default="false" />
  <arg name="params_file" default="${find-pkg-share minibot_navigation2}/params/params.yaml" />
  <arg name="autostart" default="true" />
  <arg name="container_name" default="nav2_container" />
  <arg name="use_composition" default="true" />
  <arg name="use_respawn" default="false" />
  <arg name="log_level" default="info" />
  <arg name="lifecycle_nodes_local" default="['map_server',
    'controller_server',
    'smoother_server',
    'planner_server',
    'behavior_server',
    'bt_navigator',
    'waypoint_follower',
    'localization_smoother']" />
  <arg name="lifecycle_nodes_nav"
    default="['controller_server',
    'smoother_server',
    'planner_server',
    'behavior_server',
    'bt_navigator',
    'waypoint_follower',
    'localization_smoother']" />
  <arg name="robot_name" default="minibot_1" />
  </launch>
```

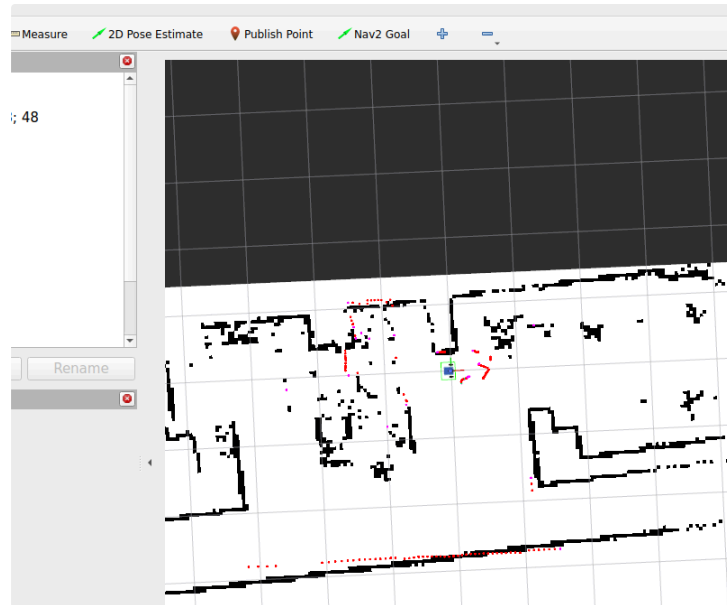
Free Fleet client parameter중 name을 argument로 받기.

```
<!-- launch the free fleet client first -->
<node name="minibot_free_fleet_client_node" pkg="free_fleet_client_ros2"
  exec="free_fleet_client_ros2" output="both">
  <param name="fleet_name" value="minibot" />
  <param name="robot_name" value="minibot_1" />
  <param name="robot_model" value="minibot" />
  <param name="level_name" value="L1" />
  <param name="dds_domain" value="43" />
  <param name="max_dist_to_first_waypoint" value="10.0" />
  <param name="map_frame" value="map" />
  <param name="robot_frame" value="base_footprint" />
  <param name="nav2_server_name" value="/navigate_to_pose" />
  <param name="use_sim_time" value="false" />
  <param name="dds_domain" value="43" />
</node>
```

```
<!-- launch the free fleet client first -->
<node name="minibot_free_fleet_client_node" pkg="free_fleet_client_ros2"
  exec="free_fleet_client_ros2" output="both">
  <param name="fleet_name" value="minibot" />
  <param name="robot_name" value="$(var robot_name)" />
  <param name="robot_model" value="minibot" />
  <param name="level_name" value="L1" />
  <param name="dds_domain" value="43" />
  <param name="max_dist_to_first_waypoint" value="10.0" />
  <param name="map_frame" value="map" />
  <param name="robot_frame" value="base_footprint" />
  <param name="nav2_server_name" value="/navigate_to_pose" />
  <param name="use_sim_time" value="false" />
  <param name="dds_domain" value="43" />
</node>
```

빌드 후 실행해보자

- ros2 launch minibot_bringup bringup_robot.launch.xml # Minibot에서 (SSH로 minibot_1접속 후)
- ros2 launch minibot_navigation2 bringup_launch.xml \
- map:=\$HOME/rmf_ws/src/rmf_demos/rmf_demos_maps/maps/pinklab_real/pinklab_L1.yaml #PC에서 실행
- rviz2 -d `ros2 pkg prefix minibot\_navigation2`/share/minibot_navigation2/rviz/nav2_view.rviz



minibot_navigation2/param에서 nav2_params.yaml을 복사한 뒤, Rviz2에서 minibot_2의 spawn 지점을 initial_pose로 지정하자. [링크4](#)

```
~/rmf_ws
> ros2 topic echo /clicked_point
header:
  stamp:
    sec: 1715238773
    nanosec: 166946016
    frame_id: map
  point:
    x: 0.07326594740152359
    y: -1.9082789421081543
    z: -0.001434326171875

minibot_navigation2
> config
> launch
> maps
> params
  nav2_params_minibot2.yaml
  nav2_params.yaml
  nav2_nav_to_pose.xml
> rviz
  CMakeLists.txt
  package.xml
  README.md
  .gitignore
  LICENSE
  README.md

recovery_alpha_fast: 0.0
recovery_alpha_slow: 0.0
resample_interval: 1
robot_model_type: "nav2_amcl::DifferentialMotionModel"
save_pose_rate: 0.5
sigma_hit: 0.2
tf_broadcast: true
transform_tolerance: 1.0
update_min_a: 0.2
update_min_d: 0.25
z_hit: 0.5
z_max: 0.99
z_rand: 0.5
z_short: 0.05
scan_topic: scan
set_initial_poses: true
initial_pose: {x: 0.07326594740152359, y: -1.9082789421081543, z: 0.0, yaw: 0.0}
```

minibot_2는 다른 ROS_DOMAIN에서 실행하자. [링크5](#)

- minibot_2의 ROS_DOMAIN_ID를 PC와 다르게 설정하고, 새 터미널에서 ROS_DOMAIN_ID를 맞춰준 후 Navigation2을 시작하자.
- params_files을 입력인자는 방금 만든 nav2_params_minibot2.yaml 를 입력하자.

```
1 # minibot에서
2 export ROS_DOMAIN_ID=33 #Minibot에서 (SSH로 minibot_2접속 후)
3 ros2 launch minibot_bringup bringup_robot.launch.xml # Minibot에서 (SSH로 minibot_2접속 후)
4
5 # PC에서
6 export ROS_DOMAIN_ID=33 # PC에서 실행
7 ros2 launch minibot_navigation2 bringup_launch.xml \
8   map:=$HOME/rmf_ws/src/rmf_demos/rmf_demos_maps/maps/pinklab_real/pinklab_L1.yaml \
9   robot_name:=minibot_2 \
10  params_file:=$HOME/rmf_ws/src/pinklab_minibot_robot/minibot_navigation2/params/nav2_params_minibot2.yaml
```

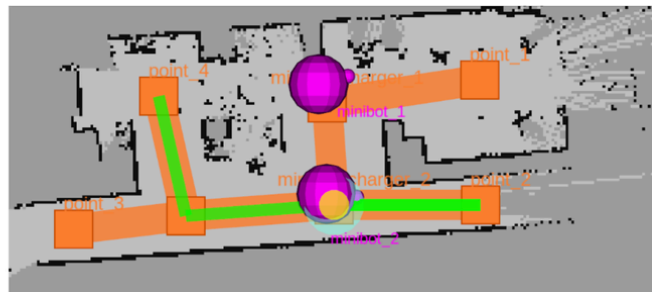
그 다음 RMF를 실행하면... [링크6](#)

```
1 ros2 launch rmf_demos pinklab_real.launch.xml
```

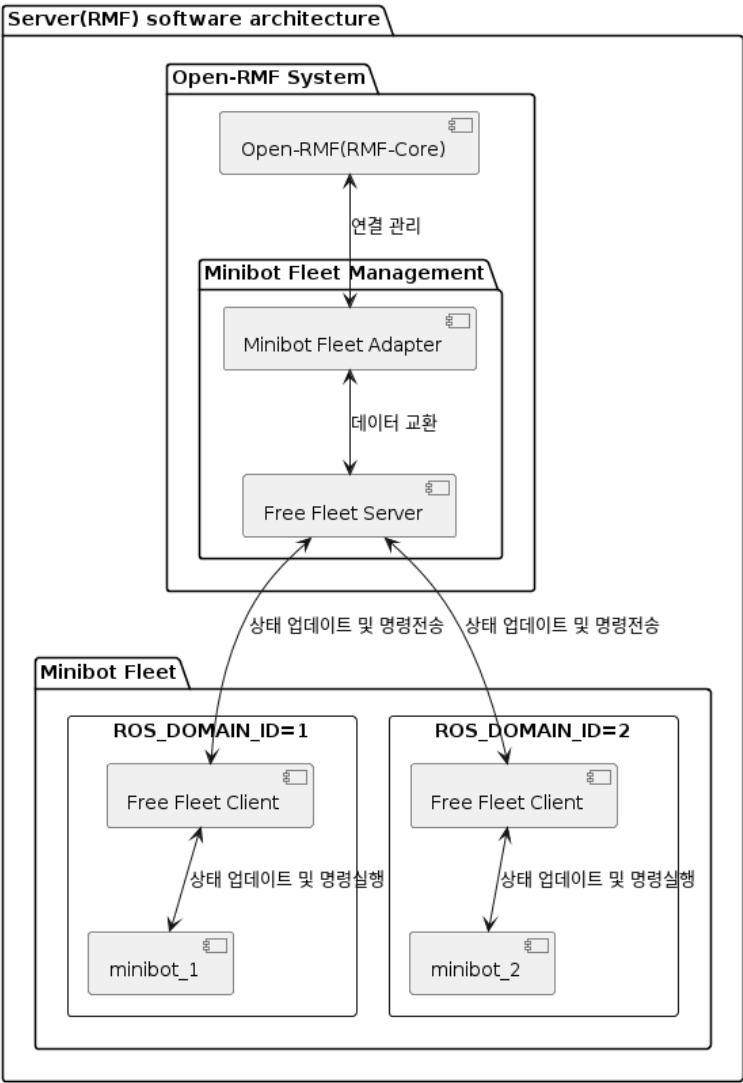


Panel로 Task를 주면 입찰받은 minibot이 움직이게 된다. [🔗](https://open-rmf.github.io/rmf-panel-js/)

<https://open-rmf.github.io/rmf-panel-js/>



같은 Fleet만 사용할 경우 RMF시스템의 소프트웨어 아키텍처 (예시) [🔗](#)



다른 Fleet를 추가하기. [↗](#)

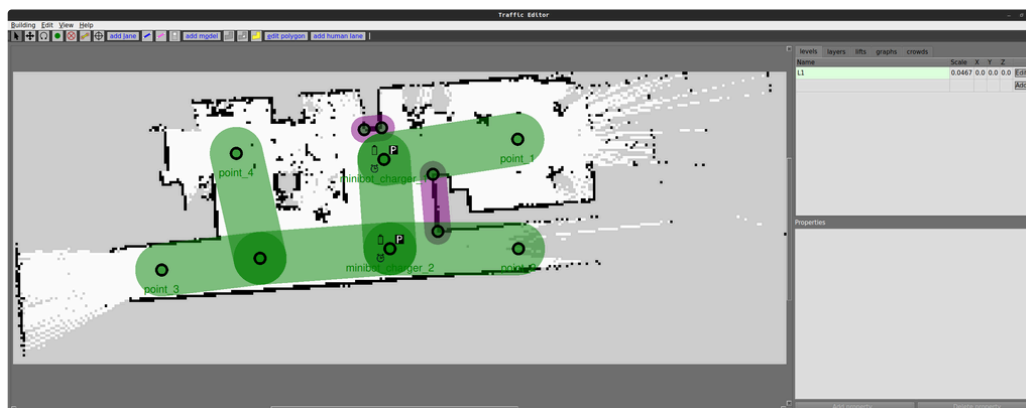
Minibot과 다른 로봇인 Turtlebot3를 RMF시스템에 추가해보자. [↗](#)



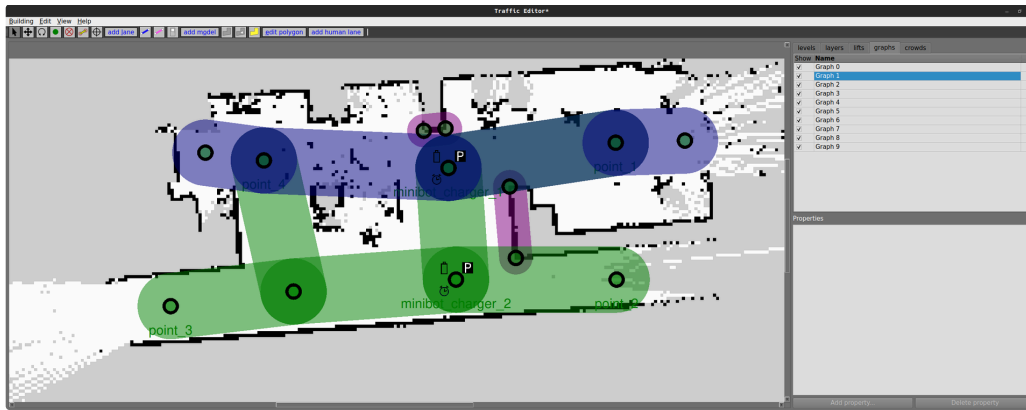
Turtlebot3 ROS2 PKG 다운로드 [↗](#)

```
1 cd ~/rmf_ws/src/  
2 git clone -b humble-devel https://github.com/ROBOTIS-GIT/DynamixelSDK.git  
3 git clone -b humble-devel https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git  
4 git clone -b humble-devel https://github.com/ROBOTIS-GIT/turtlebot3.git  
5 cd ~/rmf_ws  
6 colcon build --symlink-install  
7 echo 'export TURTLEBOT3_MODEL=burger' >> ~/.bashrc  
8 source ~/.bashrc
```

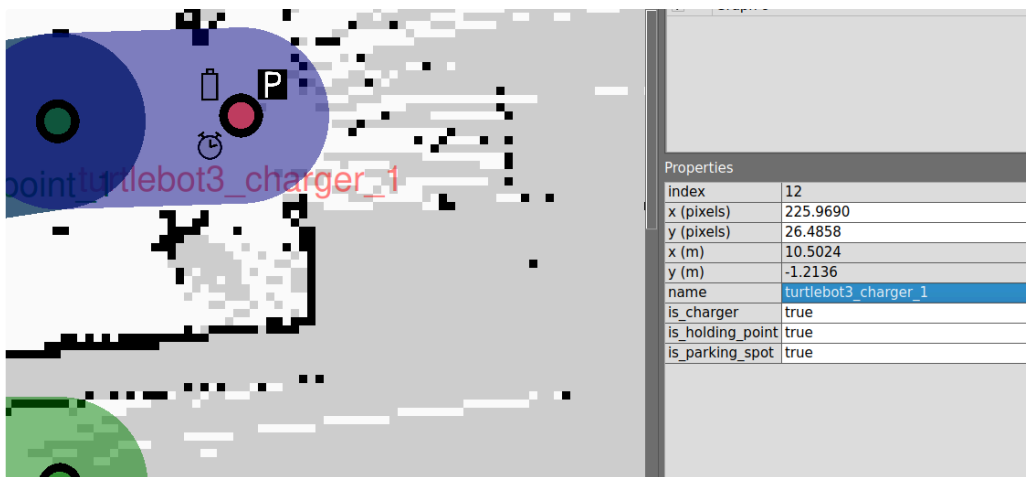
Traffic editor를 열고... [↗](#)



Graph 1번으로 Minibot Fleet와는 다른 경로를 지정하자. [링크4](#)



Spawn 지점과 관련된 설정을 하고, 저장 및 종료. [링크5](#)

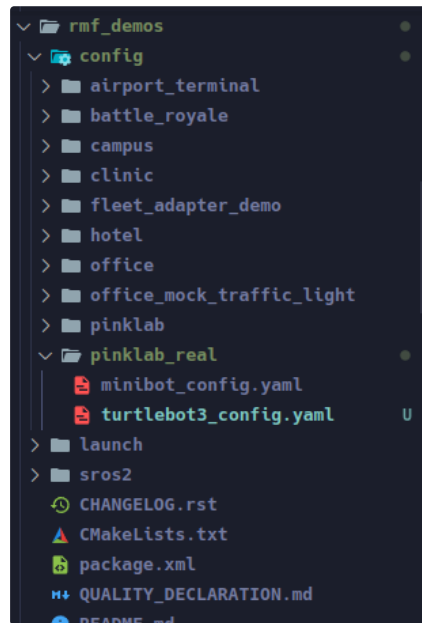


그 다음 Fleet adapter를 구축해야한다. [링크6](#)

- minibot fleet adapter와 다른점은 turtlebot fleet adapter는 배터리정보를 포함한다.
- 그 외에는 똑같다..

```
1 cd ~/rmf_ws
2 git clone https://github.com/pinklabstudio/turtlebot3_fleet_adapter.git -b main
3 colcon build --symlink-install
```

Turtlebot3의 RMF정보를 작성할 yaml파일을 만들고... [링크5](#)



Turtlebot3의 위치정보, 개체, 좌표변환을 작성. [링크6](#)

```
rmf fleet:
  name: "turtlebot3"
  fleet_manager:
    prefix: "http://127.0.0.1:22017"
    ip: "127.0.0.1"
    port: 22017
    user: "some_user"
    password: "some_password"
  limits:
    linear: [0.5, 0.75] # velocity, acceleration
    angular: [0.6, 2.0] # velocity, acceleration
  profile: # Robot profile is modelled as a circle
    footprint: 0.3 # radius in m
    vicinity: 0.5 # radius in m
    reversible: True # whether robots in this fleet can reverse
  battery system:
    voltage: 12.0 # V
    capacity: 24.0 # Ahr
    charging_current: 5.0 # A
  mechanical system:
    mass: 20.0 # kg
    moment of inertia: 10.0 # kgm^2
    friction coefficient: 0.22
  ambient system:
    power: 20.0 # W
  tool system:
    power: 0.0 # W
  recharge_threshold: 0.10 # Battery level below which robots in the fleet should be recharged
  soc: 1.0 # Battery level to which robots in this fleet should be recharged
  publish fleet state: 10.0 # Publish frequency for fleet state, or account for battery drain: True
  task capabilities: # Specify the types of RMF Tasks that robots in this fleet can perform
    loop: True
    delivery: False
    clean: False
    finishing request: "park" # [park, charge, nothing]
  actions: ["teleop"]
  responsive_wait: True # Should responsive wait be on/off for the

# turtlebot3 CONFIG =====
robots:
  turtlebot3_1:
    robot_config:
      max_delay: 10.0 # allowed seconds of delay of the robot
    rmf_config:
      robot_state_update_frequency: 1.0
      charger:
        waypoint: "turtlebot3_charger_1"
      start:
        map_name: "L1"
        waypoint: "turtlebot3_charger_1"
        orientation: 0.0 # radians

# TRANSFORM CONFIG =====
# For computing transforms between Robot and RMF coordinates

# Optional
reference_coordinates:
  rmf: [[0.0, 0.0],
        [14.9834, 0.0],
        [14.9834, -5.5079],
        [0.0, -5.5079]]
  robot: [[-7.311037063598633, 1.4209141731262207],
           [8.718361854553223, 1.4333285093307495],
           [8.712226867675781, -4.453343868255615],
           [-7.3213982582092285, -4.454360485076904]]
```

RMF시스템 시작 시 turtlebot3 Fleet adapter를 같이 시작하도록 설정. [링크7](#)

- RMF시스템을 시작하는 파일은 rmf_demos/launch/pinklab_real.launch.xml에 있다.

```
<!-- turtlebot3 fleet adapter -->
<include file="$(find-pkg-share turtlebot3_fleet_adapter)/launch/fleet_turtlebot3_adapter.launch.xml">
  <arg name="use_sim_time" value="$(var use_sim_time)" />
  <arg name="nav_graph_file"
        value="$(find-pkg-share rmf_demos_maps)/maps/pinklab_real/nav_graphs/1.yaml" />
  <arg name="config_file"
        value="$(find-pkg-share rmf_demos)/config/pinklab_real/turtlebot3_config.yaml" />
</include>
```


RMF시스템에 turtlebot3의 navigation2정보를 취합할 Free Fleet Server코드 작성. [링크8](#)

```
<!-- launch the free fleet server -->
<node pkg="free_fleet_server_ros2"
  exec="free_fleet_server_ros2"
  name="turtlebot3_fleet_server_node"
  output="both">

  <param name="fleet_name" value="turtlebot3" />

  <param name="fleet_state_topic" value="turtlebot3/fleet_states" />
  <param name="mode_request_topic" value="turtlebot3/robot_mode_requests" />
  <param name="path_request_topic" value="turtlebot3/robot_path_requests" />
  <param name="destination_request_topic" value="turtlebot3/robot_destination_requests" />

  <param name="dds_domain" value="44" />
  <param name="dds_robot_state_topic" value="robot_state" />
  <param name="dds_mode_request_topic" value="mode_request" />
  <param name="dds_path_request_topic" value="path_request" />
  <param name="dds_destination_request_topic" value="destination_request" />

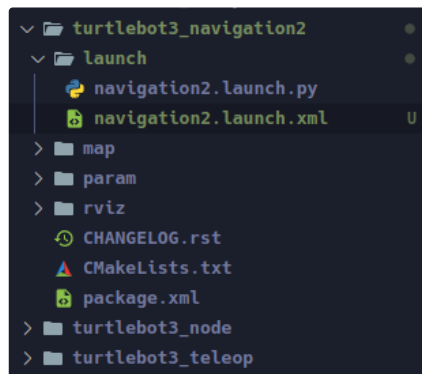
  <param name="update_state_frequency" value="20.0" />
  <param name="publish_state_frequency" value="2.0" />

  <param name="translation_x" value="0.0" />
  <param name="translation_y" value="0.0" />
  <param name="rotation" value="0.0" />
  <param name="scale" value="1.0" />

</node>
```

Navigation2을 시작하면서, Free fleet client를 같이 시작하기 위해 파일을 한 개 만들고...

- turtlebot3 pkg중 turtlebot3_navigation2의 navigation2.launch.xml을 만들자.



Navigation2과 Free Fleet client 코드를 작성. [링크9](#)

```
<?xml version='1.0'?>
<launch>
  <!-- Arguments -->
  <arg name="map" default="" />

  <!-- launch the navigation2 stack -->
  <include file="$(find-pkg-share turtlebot3_navigation2)/launch/navigation2.launch.py">
    <arg name="map" value="$(var map)" />
  </include>

  <!-- launch the free fleet client first -->
  <node name="turtlebot3_free_fleet_client_node" pkg="free_fleet_client_ros2"
    exec="free_fleet_client_ros2" output="both">
    <param name="fleet_name" value="turtlebot3" />
    <param name="robot_name" value="turtlebot3_1" />
    <param name="robot_model" value="turtlebot3" />
    <param name="level_name" value="L1" />
    <param name="dds_domain" value="44" />
    <param name="max_dist_to_first_waypoint" value="10.0" />
    <param name="map_frame" value="map" />
    <param name="robot_frame" value="base_footprint" />
    <param name="nav2_server_name" value="/navigate_to_pose" />
    <param name="use_sim_time" value="False" />
  </node>
</launch>
```

빌드한 뒤, 먼저 Minibot을 다 실행하고...

```
1 ros2 launch minibot_bringup bringup_robot.launch.xml # Minibot에서 (SSH로 minibot_1접속 후)
2 ros2 launch minibot_navigation2 bringup_launch.xml \
3   map:=$HOME/rmf_ws/src/rmf_demos/rmf_demos_maps/maps/pinklab_real/pinklab_L1.yaml #PC에서 실행
4 rviz2 -d `ros2 pkg prefix minibot_navigation2`/share/minibot_navigation2/rviz/nav2_view.rviz
```

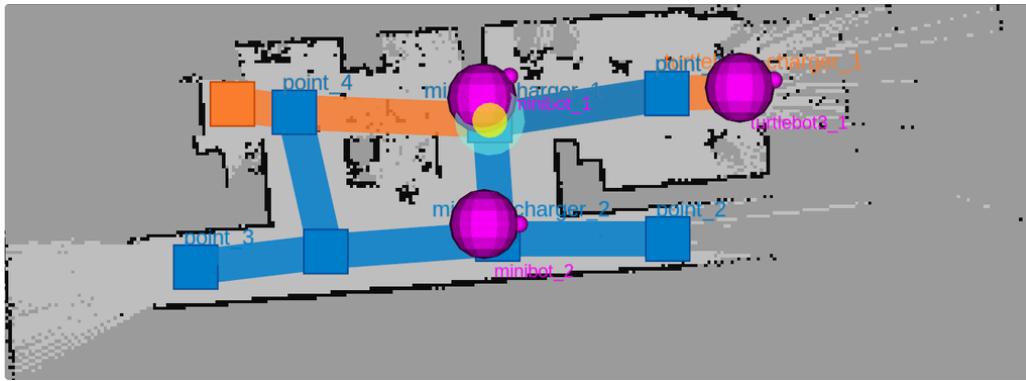
```
1 # minibot에서
2 export ROS_DOMAIN_ID=33 #Minibot에서 (SSH로 minibot_2접속 후)
3 ros2 launch minibot_bringup bringup_robot.launch.xml # Minibot에서 (SSH로 minibot_2접속 후)
4
5 # PC에서
6 export ROS_DOMAIN_ID=33 # PC에서 실행
7 ros2 launch minibot_navigation2 bringup_launch.xml \
8   map:=$HOME/rmf_ws/src/rmf_demos/rmf_demos_maps/maps/pinklab_real/pinklab_L1.yaml \
9   robot_name:=minibot_2 \
10  params_file:=$HOME/rmf_ws/src/pinklab_minibot_robot/minibot_navigation2/params/nav2_params_minibot2.yaml
```

Turtlebot3 실행

```
1 # turtlebot3에서
2 export ROS_DOMAIN_ID=34 #turtlebot에서 (SSH로 turtlebot3_1 접속 후)
3 ros2 launch turtlebot3_bringup robot.launch.py # turtlebot에서 (SSH로 turtlebot3_1접속 후)
4 # PC에서
5 export ROS_DOMAIN_ID=34 #turtlebot에서 (SSH로 turtlebot3_1 접속 후)
6 ros2 launch turtlebot3_navigation2 navigation2.launch.xml \
7   map:=$HOME/rmf_ws/src/rmf_demos/rmf_demos_maps/maps/pinklab_real/pinklab_L1.yaml
```

그 다음 RMF를 실행하면

```
1 ros2 launch rmf_demos pinklab_real.launch.xml
```



타 기종을 포함하는 RMF 시스템의 소프트웨어 아키텍처 예시는 다음과 같다. [↗](#)

